

Reviewer Pack — Minimal Rank of Affine Sieve Representations vs DPLL Search ...

Entry df7fa212f36e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 13:22:37 UTC

Minimal Rank of Affine Sieve Representations vs DPLL Search Tree Width

Entry ID: df7fa212f36e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 13:22:37 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

[“For every Boolean satisfiability problem (SAT) instance with n variables, there exists an affine sieve representation of the instance’s clause-indicator polynomial such that its minimal rank is upper-bounded by a function $f(n) = \Theta(\sqrt{n} \log n)$.”, ‘Furthermore, for all instances with a DPLL search tree width greater than some constant C , there exists an affine sieve representation with a minimal rank that exceeds $f(n)$.’, ‘The minimal rank of an affine sieve representation provides a lower bound on the number of different variable assignments needed to solve the SAT instance.’]

Rationale (proposer’s reasoning):

[‘Affine sieve representations are a tool from algebraic combinatorics that can encode polynomial equations over fields, potentially capturing non-trivial structures in the Boolean satisfiability problem.’, ‘Previous work has shown that certain complexity measures on DPLL search trees can provide insights into the structure of SAT instances. Linking these two areas might reveal new properties about the hardness of SAT problems.’, ‘The conjecture suggests a relationship between the algebraic encoding and the structural complexity of the instance, which could potentially lead to new algorithmic approaches or lower bounds.’]

Taxonomy category: AFFINE_SIEVE_REPRESENTATIONS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dcda43b59ec35bfc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean minimal rank of affine sieve representations across 30 seeds does not exceed $f(n) = \Theta(\sqrt{n} \log n)$, and no seed shows a minimal rank

exceeding $f(n)$. Falsified if any seed demonstrates a minimal rank exceeding $f(n)$ or the mean minimal rank exceeds $f(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def compute_clause_indicator_polynomial(clauses, n):
        if n > 20:
            return None # Avoid large memory usage for n > 20
        polynomial = [0] * (2 ** n)
        for clause in clauses:
            product = 1
            for literal in clause:
                index = literal if literal >= 0 else -(literal + 1)
                product *= (-1) ** literal * (1 - x[index])
            polynomial += product
        return polynomial

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i
            for j in range(i + 1, rows):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            pivot = matrix[i][i]
            for j in range(cols):
```

```

        matrix[i][j] /= pivot

    for j in range(rows):
        if i != j:
            factor = matrix[j][i]
            for k in range(cols):
                matrix[j][k] -= factor * matrix[i][k]

    rank = 0
    for row in matrix:
        if any(row):
            rank += 1
    return rank

def dpll_search_tree_width(clauses, assignment):
    stack = [(clauses, assignment)]
    while stack:
        clauses, assignment = stack.pop()
        if not clauses:
            return len(assignment)
        clause = next(c for c in clauses if any(lit in assignment or -lit in assignment for lit in c))
        literals = [lit for lit in clause if lit not in assignment and -lit not in assignment]
        for literal in literals:
            new_assignment = assignment + [literal]
            new_clauses = [c for c in clauses if literal not in c and -literal not in c]
            stack.append((new_clauses, new_assignment))
    return float('inf')

n = random.randint(5, 40)
num_clauses = random.randint(n, n * (n + 1) // 2)
clauses = []
for _ in range(num_clauses):
    clause = [random.choice([-1, 1]) * (i + 1) for i in range(random.randint(1, n))]
    clauses.append(clause)

polynomial = compute_clause_indicator_polynomial(clauses, n)
if polynomial is None:
    return {
        "metric_name": "minimal_rank",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

x = [Fraction(0, 1)] * n
matrix = []
for i in range(2 ** n):
    row = []
    for j in range(n):
        if (i >> j) & 1:
            row.append(-x[j])
        else:
            row.append(x[j])
    matrix.append(row)

```

```

rank = gaussian_elimination(matrix)
dpll_width = dpll_search_tree_width(clauses, [])

return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank <= math.sqrt(n) * math.log(n),
    "counterexample": "" if rank <= math.sqrt(n) * math.log(n) else f"rank={rank} > 0(√{n} log
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank exceeds 0(√n log n)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'minimal_rank', 'metric_value': inf, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': inf, 'instances_tested': 1, 'conjecture_holds': False}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b847d09f.py", line 117, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b847d09f.py", line 79, in run_trial
    polynomial = compute_clause_indicator_polynomial(clauses, n)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b847d09f.py", line 29, in compute_clause_indicator_polynomial
    product *= (-1) ** literal * (1 - x[index])
                                   ^

```

NameError: cannot access free variable 'x' where it is not associated with a value in enclosing scope

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15086
2	preregistration	ollama_remote	glm4:latest	0	0	9456
3	novelty	ollama_remote	glm4:latest	0	0	8859
4	novelty	ollama_remote	glm4:latest	0	0	8851
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19761
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11194
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15281
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14260
9	judge	ollama_remote	glm4:latest	0	0	20897

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123645 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/df7fa212f36e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/df7fa212f36e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/df7fa212f36e.tar.gz (if generated)